

Fase 2

María Alexandra Jiménez Suárez

Juan José Álvarez Restrepo

Carolina Ramírez Lotero

Natalia Giraldo Morales

Universidad Pontificia Bolivariana

Facultad de Ingeniería

Cesar Augusto López Gallego

10/08/2026

Fase 2 – Análisis de solicitudes de cambio

Impacto de las solicitudes de cambio en el dominio

SC-1

- **Nuevo concepto:** Cosméticos y productos comestibles.
- **Parte del dominio afectada:** Jerarquía de Producto y mecanismos de creación de productos.
- **Impacto sobre el modelo actual:** Producto ya incluye atributos generales de los elementos que se venden, como el nombre, el precio y el stock. No obstante, la jerarquía vigente se especializa en medicamentos y tanto ProductoFactory como ServicioProducto.CargarDesdeArchivo() interactúan directamente con las distintas formas de medicamentos disponibles. Los nuevos productos no pueden incorporarse como especializaciones de Medicamento, porque no representan productos farmacéuticos.

Para el diseño se consideraría extender Producto mediante nuevas especializaciones independientes de Medicamento, manteniendo separado el conjunto de elementos farmacéuticos de los demás tipos de productos, sin romper o modificar la jerarquía ya existente.

SC-2

- **Nuevo concepto:** Servicios de farmacia como inyectología, curaciones y cambio de vendajes
- **Parte del dominio afectada:** Abstracción Producto, ServicioProducto, Movimiento y los eventos asociados a stock y vencimiento.
- **Impacto sobre el modelo actual:** El modelo actual emplea el Producto para representar elementos que tienen precio, stock, stock mínimo y fecha de caducidad. A pesar de que un servicio puede tener un precio y ser parte de una transacción comercial, no tiene que compartir necesariamente las propiedades físicas de un producto. Por lo tanto, incluirlo en la jerarquía actual implicaría examinar qué propiedades y comportamientos de Producto son verdaderamente pertinentes para el nuevo concepto.

El diseño deberá establecer una separación conceptual entre productos físicos y servicios, evitando extender la jerarquía de Producto únicamente para reutilizar atributos o comportamientos que no representan al servicio.

SC-3

- **Nuevo concepto:** Convenio y entidades asociadas
- **Parte del dominio afectada:** Cliente, reglas de descuento y las clases relacionadas con las operaciones comerciales.
- **Impacto sobre el modelo actual:** El dominio actual representa al Cliente y sus puntos, pero no contempla relaciones con entidades externas ni condiciones comerciales asociadas a convenios. Aunque existen IDescuento y ServicioDescuento, estas abstracciones no están integradas al modelo actual. El convenio representa una condición comercial del cliente y no un nuevo tipo de cliente.

El diseño deberá incorporar el concepto de convenio de forma independiente a la identidad de Cliente, así se podrá vincular con diversas clases de entidades y plasmar las reglas comerciales pertinentes sin establecer una jerarquía de clientes basada en el tipo de convenio.

Funcionalidades que podrían romperse

En esta fase se revisó el comportamiento actual del sistema frente a las tres solicitudes de cambio planteadas, con el fin de identificar qué funcionalidades podrían verse afectadas si se incorporan los nuevos requerimientos.

SC-1 - Nuevos tipos de productos

La solicitud plantea que la farmacia no solo maneje medicamentos, sino también cosméticos y productos comestibles como gaseosas, agua, helados y snacks.

Al revisar el código se encontró que el sistema actualmente trabaja los productos a partir de una clase abstracta Producto, que tiene nombre, precio, stock, stock mínimo y fecha de vencimiento. Esto permite que la venta y algunas funcionalidades generales trabajen sobre Producto y no directamente sobre Medicamento.

Sin embargo, hay comportamientos que podrían verse afectados con los nuevos tipos de productos.

```
public abstract class Producto
{
    8 references
    public string Nombre { get; set; }
    5 references
    public decimal Precio { get; set; }
    7 references
    public int Stock { get; set; }
    2 references
    public int StockMinimo { get; set; }
    2 references
    public DateTime FechaVencimiento { get; set; }

    1 reference
    protected Producto(string nombre,
        decimal precio,
        int stock,
        int stockMinimo,
        DateTime fechaVencimiento)
    {
        Nombre = nombre;
        Precio = precio;
        Stock = stock;
        StockMinimo = stockMinimo;
        FechaVencimiento = fechaVencimiento;
    }
}
```

Carga y creación de productos: actualmente los productos que se cargan desde productos.txt se crean como MedicamentoCapsula. Además, la ProductoFactory tiene métodos para crear medicamentos en cápsula y líquidos. Por lo tanto, la carga y creación actual no contempla directamente cosméticos ni productos comestibles.

```

public static MedicamentoCapsula CrearCapsula(
    string nombre,
    decimal precio,
    int stock,
    Laboratorio laboratorio)
{
    return new MedicamentoCapsula(
        nombre,
        precio,
        stock,
        5,
        DateTime.Now.AddMonths(6),
        laboratorio,
        TipoRelleno.Gel);
}
0 references
public static MedicamentoLiquido CrearLiquido(
    string nombre,
    decimal precio,
    int stock,
    Laboratorio laboratorio)
{
    return new MedicamentoLiquido(
        nombre,
        precio,
        stock,
        5,
        DateTime.Now.AddMonths(12),
        laboratorio,
        TipoRelleno.Liquido);
}

```

Alertas de vencimiento: todos los productos tienen una fecha de vencimiento y ServicioProducto genera una alerta cuando faltan 30 días o menos. Esto podría generar un comportamiento incorrecto para algunos nuevos productos, ya que actualmente se aplica la misma regla de vencimiento a cualquier objeto que sea Producto.

```

public void VerificarVencimiento()
{
    foreach (var producto in productos)
    {
        int dias =
            (producto.FechaVencimiento -
             DateTime.Now).Days;

        if (dias <= 30)
        {
            EventoVencimiento
                .Disparar(producto);
        }
    }
}

```

Alertas de stock: esta funcionalidad podría continuar funcionando para los nuevos productos porque los cosméticos y los alimentos también manejan cantidades disponibles. Sin embargo, actualmente la regla es general y no diferencia entre los diferentes tipos de productos.

```

public class EventoStockMinimo
{
    1 reference
    public delegate void DelegadoStock(
        string mensaje);

    2 references
    public event DelegadoStock? StockMinimo;

    1 reference
    public void Disparar(
        Producto producto)
    {
        StockMinimo?.Invoke(
            $"ALERTA: stock mínimo de {producto.Nombre}");
    }
}

```

Venta: esta funcionalidad tiene menos riesgo que las anteriores, porque la venta trabaja con la abstracción Producto: busca el producto, disminuye su stock y registra el movimiento. Por esta razón, no está limitada directamente a los medicamentos. El posible problema estaría más relacionado con las características específicas que puedan necesitar los nuevos tipos de productos.

```

var productoVenta =
    servicioProducto
        .ObtenerProductos()
        .FirstOrDefault(p =>
            p.Nombre.ToLower()
                .Contains(
                    nombreVenta.ToLower()));

if (productoVenta != null)
{
    Console.Write(
        "Cantidad: ");

    int cantidad =
        int.Parse(
            Console.ReadLine());

    productoVenta.Stock -=
        cantidad;

    Movimiento venta =
        new Movimiento(
            DateTime.Now,
            cantidad,
            "Venta",
            productoVenta);
}

```

En general, para SC-1 los mayores riesgos están en la **carga y creación de productos y en las reglas de vencimiento**, mientras que la venta y el manejo de stock ya están contruidos sobre la clase general Producto.

SC-2 - Incorporación de servicios

La segunda solicitud plantea que la farmacia también ofrecerá servicios como inyectología, cambio de vendajes y curaciones básicas.

Aquí se encontró un mayor riesgo de afectar funcionalidades existentes, porque el sistema actual está construido principalmente alrededor del concepto de Producto.

Registrar venta: actualmente una venta recibe un producto y una cantidad, descuenta el stock y crea un movimiento. Un servicio como una inyectología no funciona de la misma manera, por lo que la lógica actual de venta no representa directamente este tipo de operación.

```
var productoVenta =  
    servicioProducto  
        .ObtenerProductos()  
        .FirstOrDefault(p =>  
            p.Nombre.ToLower()  
            .Contains(  
                nombreVenta.ToLower()));
```

Listado y búsqueda de productos: las opciones actuales trabajan sobre la lista de Producto. Por esto, un servicio no aparecería en el listado ni en la búsqueda de productos utilizando el comportamiento actual.

Alertas de stock: ServicioProducto revisa el stock de cada producto y genera una alerta cuando llega al stock mínimo. Esto tiene sentido para medicamentos o productos físicos, pero no para un servicio como una curación o una inyectología.

```
public void VerificarStock()  
{  
    foreach (var producto in productos)  
    {  
        if (producto.Stock <= producto.StockMinimo)  
        {  
            EventoStock.Disparar(producto);  
        }  
    }  
}
```

Alertas de vencimiento: la verificación utiliza la FechaVencimiento del producto. Un servicio no tiene una fecha de vencimiento como la que maneja actualmente el sistema, por lo que esta funcionalidad no se podría aplicar de la misma forma.

```
public void VerificarVencimiento()  
{  
    foreach (var producto in productos)  
    {  
        int dias =  
            (producto.FechaVencimiento -  
             DateTime.Now).Days;  
  
        if (dias <= 30)  
        {  
            EventoVencimiento  
                .Disparar(producto);  
        }  
    }  
}
```

Registro de movimientos: la clase Movimiento guarda la fecha, cantidad, tipo y un Producto. Por esto, una operación relacionada con la prestación de un servicio no queda representada directamente con la estructura actual.

```

public class Movimiento
{
    1 reference
    public DateTime Fecha { get; set; }
    1 reference
    public int Cantidad { get; set; }
    2 references
    public string Tipo { get; set; }
    1 reference
    public Producto Producto { get; set; }
}

```

El movimiento está relacionado directamente con un Producto, por lo que actualmente no existe una forma directa de representar la prestación de un servicio como una inyectología o una curación.

Por lo anterior, SC-2 es el cambio que presenta mayor riesgo de afectar varias funcionalidades existentes, principalmente porque los servicios no siguen las mismas características que los productos físicos.

SC-3 - Convenios con entidades

La tercera solicitud plantea que la farmacia pueda manejar convenios con empresas, bancos, cooperativas, mutuales, universidades y colegios, para ofrecer descuentos y créditos.

En este caso, las funcionalidades que podrían verse afectadas están principalmente relacionadas con los clientes y las ventas.

Registrar venta: actualmente la venta solo recibe el producto y la cantidad, descuenta el stock y registra el movimiento. No se maneja información sobre descuentos, convenios o créditos. Por ejemplo, si un cliente tiene un convenio con una empresa, el comportamiento actual no contempla que ese convenio pueda modificar el valor de la compra.

Acumular puntos: actualmente los puntos se suman directamente al cliente:

Puntos nuevos = puntos anteriores + puntos ingresados

Con los convenios podrían existir condiciones diferentes para los beneficios de cada entidad, por lo que esta funcionalidad podría verse afectada dependiendo de las reglas que se definan para los convenios.

```

var clientePuntos =
    servicioCliente
        .ObtenerClientes()
        .FirstOrDefault(c =>
            c.Nombre.ToLower()
                .Contains(
                    nombreCliente.ToLower()));

```

```
servicioCliente
    .AcumularPuntos(
        clientePuntos,
        puntos);
```

Estas dos capturas evidencian que actualmente los puntos se manejan de manera directa y que la operación no contempla ningún convenio.

Información del cliente: actualmente Cliente hereda de Persona y agrega únicamente los puntos. Persona maneja nombre, cédula, teléfono y correo. No se encuentra información relacionada con la entidad del convenio, el tipo de convenio, descuentos o crédito. Por esto, el comportamiento actual del cliente no permite representar completamente este nuevo escenario.

```
public class Cliente : Persona
{
    4 references
    public int Puntos { get; set; }
```

Registro de movimientos: un movimiento actualmente almacena fecha, cantidad, tipo y producto. No contiene información sobre descuentos, créditos o convenios, por lo que una venta realizada bajo un convenio no podría quedar representada completamente con la información actual.

Búsqueda de clientes: la búsqueda actual permite encontrar clientes por nombre y acepta coincidencias parciales. Si el nuevo requerimiento necesita identificar también la entidad con la que está asociado el cliente, la búsqueda actual podría no ser suficiente.

Login: no se encontró una afectación directa en esta funcionalidad. El login trabaja con Usuario y sus datos de autenticación, por lo que los convenios no cambian directamente este comportamiento.

Conclusión del análisis

Después de revisar las tres solicitudes, se encontró que no todas afectan el sistema de la misma manera.

En **SC-1**, la venta y el manejo general de stock tienen una base que podría permitir trabajar con nuevos productos, pero la carga, creación y algunas reglas como el vencimiento están orientadas al comportamiento actual de los medicamentos.

En **SC-2**, se encuentra un mayor riesgo, porque varias funcionalidades actuales dependen directamente de que la información corresponda a un Producto. Los servicios no tienen las mismas características de stock y vencimiento y tampoco están contemplados en el registro de movimientos.

En **SC-3**, los principales riesgos están en las ventas, los puntos, la información de los clientes y los movimientos, ya que actualmente el sistema no contempla convenios, descuentos ni créditos.

Estos son los comportamientos actuales que se consideran importantes revisar antes de realizar cualquier cambio al sistema.

SC-1

Archivo	Tipo de cambio	Evidencia
BibFarmacia/Servicios/ServicioProducto.cs, líneas 99–107	Modificar	CargarDesdeArchivo() construye siempre un MedicamentoCapsula, sin condición alguna. Hay que enseñarle a reconocer y crear los nuevos tipos. <pre> MedicamentoCapsula medicamento = new MedicamentoCapsula(datos[0], decimal.Parse(datos[1]), int.Parse(datos[2]), int.Parse(datos[3]), DateTime.Parse(datos[4]), laboratorio, Enum.TipoRelleno.Gel); productos.Add(medicamento); </pre>
Cosmetico.cs	Crear	No existe ninguna clase para esta familia.
ProductoComestible.cs	Crear	No existe ninguna clase para esta familia.

Notas:

- VerificarStock() (línea 47-57) y VerificarVencimiento() (línea 59-73) iteran sobre List<Producto> usando solo propiedades de la clase base, funcionan para cosméticos y comestibles sin cambiar una sola línea.
- el archivo de datos productos.txt tampoco tiene una columna que indique el tipo de producto, habría que rediseñar su formato también.

SC-2

Archivo	Tipo de cambio	Evidencia
Servicio.cs	Crear	Para reutilizar la venta y el registro de movimientos actuales, esta clase queda obligada a heredar de Producto, aunque un servicio no tiene stock físico ni fecha de vencimiento real.
BibFarmacia/Servicios/ServicioProducto.cs, líneas 47–73 y 99–118	Modificar	VerificarStock(), VerificarVencimiento() y CargarDesdeArchivo() las tres, no una, necesitan excluir o tratar distinto a los servicios para no generar alertas falsas de "stock mínimo" o "por vencer" sobre una inyectología.

		<pre> 2 referencias public void VerificarStock() { foreach (var producto in productos) { if (producto.Stock <= producto.StockMinimo) { MedicamentoCapsula medicamento = new MedicamentoCapsula(datos[0], decimal.Parse(datos[1]), int.Parse(datos[2]), int.Parse(datos[3]), DateTime.Parse(datos[4]), laboratorio, Enum.TipoRelleno.Gel); productos.Add(medicamento); } return "Productos cargados"; } catch (Exception ex) { return ex.Message; } } </pre>
--	--	---

```

public abstract class Producto
{
    public string Nombre { get; set; }
    public decimal Precio { get; set; }
    public int Stock { get; set; }
    public int StockMinimo { get; set; }
    public DateTime FechaVencimiento { get; set; }

    protected Producto(string nombre,
        decimal precio,
        int stock,
        int stockMinimo,
        DateTime fechaVencimiento)
    {
        Nombre = nombre;
        Precio = precio;
        Stock = stock;
        StockMinimo = stockMinimo;
        FechaVencimiento = fechaVencimiento;
    }
}

```

El riesgo esta en cuando se vaya a ejecutar desde la consola. Si Servicio hereda de Producto, el constructor lo obliga a inventar una FechaVencimiento y un StockMinimo que no significan nada.

SC-3

Archivo	Tipo de cambio	Evidencia
BibFarmacia/Clases/Cliente.cs	Modifica r	Solo tiene Puntos. No hay ningún campo para representar a qué entidad está afiliado el cliente.

		<pre> public class Cliente : Persona { public int Puntos { get; set; } public Cliente(string nombre, string cedula, string telefono, string correo) : base(nombre, cedula, telefono, correo) { Puntos = 0; } public void AcumularPuntos(int puntos) { Puntos += puntos; } } </pre>
BibFarmacia/Servicios/ServicioCliente.cs	Modificar	AcumularPuntos() suma puntos con una única regla fija (Puntos += puntos); Cargar() no sabe leer ningún dato de convenio desde archivo. <pre> public void AcumularPuntos(Cliente cliente, int puntos) { cliente.Puntos += puntos; EventoPuntos.Disparar(cliente.Nombre, puntos); } </pre>
BibFarmacia/Clases/Movimiento.cs	Modificar	No existe ningún campo para descuento aplicado ni crédito otorgado en un movimiento. <pre> public class Movimiento { public DateTime Fecha { get; set; } public int Cantidad { get; set; } public string Tipo { get; set; } public Producto Producto { get; set; } public Movimiento(DateTime fecha, int cantidad, string tipo, Producto producto) { Fecha = fecha; Cantidad = cantidad; Tipo = tipo; Producto = producto; } } </pre>
BibFarmacia/Servicios/ServicioDescuento.cs, línea 13–16	Modificar	Ya existe, pero calcula un único 10% fijo para todo el mundo (precio * 0.10m) no diferencia entre empresa, banco, cooperativa, mutual, universidad

		o colegio, que es justo lo que pide la solicitud. <pre>public class ServicioDescuento : IDescuento { public decimal CalcularDescuento(decimal precio) { return precio * 0.10m; } }</pre>
Convenio.cs	Crear	No existe ninguna clase para representar la entidad ni sus reglas.

Nota: IDescuento y ServicioDescuento ya existen en el código (confirmado: no se referencian desde ningún otro punto del sistema, es código huérfano). Es decir, alguien ya empezó a construir esta funcionalidad y la dejó a medias. Implementar SC-3 no es partir de cero: es terminar algo que ya se pagó parcialmente.

A grandes rasgos...

- SC-3 es la más cara en volumen de trabajo (5 archivos), pero es la más segura de las tres: cada cambio es *aditivo* (se agregan campos y reglas), no rompe el modelo de dominio existente.
- SC-2 es la más barata en volumen (2 archivos) pero la más peligrosa: obliga a mentirle al sistema (un servicio fingiendo ser un producto con stock), y esa mentira compila y corre sin ningún error visible.
- SC-1 es la más equilibrada: toca poco, no rompe nada de lo que ya funciona, y confirma que la única parte del sistema bien diseñada hoy es la que opera sobre la abstracción Producto.